

AI 활용 기술 문서 — 냥 아레나 (Nyang Arena)

NAN 2026 사전과제 제출물 · 2026-08-05

1. 요약

이 게임에서 AI는 **게임 규칙을 만드는 자리에** 있다. 대사 생성이나 이미지 장식이 아니다.

매 판 적용되는 시너지 규칙 3개는 LLM이 생성한 풀에서 뽑힌다. 다만 **LLM 출력을 그대로 실행하지 않는다**. 출력을 좁은 DSL로 강제하고, 화이트리스트를 통과하지 못한 것은 폐기하고, 통과한 값도 게임이 감당하는 범위로 클램프한다. 전부 폐기돼도 프리셋으로 풀백해서 게임은 반드시 동작한다.

핵심 주장은 이것이다: **생성형 AI를 게임에 넣을 때 어려운 부분은 생성이 아니라 압축이다**. 모델이 무엇을 뱉든 런타임 제약이 깨지지 않도록 만드는 층이 실제 엔지니어링이고, 이 문서의 대부분은 그 층에 대한 것이다.

2. 구조

2.1 빌드타임 생성 / 런타임 소비 분리

[개발 시간]

[배포 후 런타임]

scripts/gen-synergies.mjs

LLM 호출 (또는 세션 생성 후보)

|

▼

src/validate/synergy-schema.ts

화이트리스트 검사 → 폐기

수치 범위 → 하드 클램프

텍스트 → 새니타이즈

id 중복 → 제거

|

▼

src/data/synergies.json —(git 커밋)→ 런에서 3개 추출

네트워크 호출 0건

비용 0원

왜 런타임 호출을 하지 않는가. 두 공모전 모두 "심사자가 별도 유료 라이선스·승인 없이 실행할 수 있어야 한다"고 요구한다. 런타임에 LLM을 부르면 (a) 심사자에게 API 키를 요구하거나 (b) 제출자 키를 클라이언트에 노출하거나 (c) 프록시 서버를 심사 기간 내내 살려 두어야 한다. 셋 다 제출물의 실행 보장을 떨어뜨린다. 그래서 생성은 개발 시간에 끝내고, 결과를 저장소에 커밋하고, 런타임은 순수 정적으로 만들었다.

검증 결과: 배포 번들에 `fetch( / XMLHttpRequest / WebSocket / 외부 URL 문자열`이 **0건**이다. 브라우저 DevTools에서도 요청 50건이 전부 동일 출처 정적 자산(HTML 1 + JS 1 + 스프라이트 48)이다.

Vite가 기본으로 넣는 `modulepreload` 폴리필에 `fetch(`가 남아 있어(실제로는 발화하지 않지만) `modulePreload: false`로 제거했다. 규칙 위반이 아니어도 감사 표면을 남길 이유가 없다.

2.2 8/26 런타임 전환 경로

같은 스키마와 검증기를 그대로 쓰면서 프록시 하나만 추가하면 런타임 생성으로 승격된다. 이때도 실패 시 `synergies.json`으로 폴백하므로 계약은 동일하다. 사전과제 단계에서는 실행 보장을 우선했다.

3. 사용한 AI 도구

도구	용도	흔적
Claude Code (claude-opus-5)	전체 구현: 설계, TypeScript 코드, 스프라이트 슬라이싱 스크립트, 검증기, 밸런스 하네스, 문서	커밋 히스토리 전체. 각 커밋에 <code>Co-Authored-By</code> 명시
Claude (동일 세션)	시너지 규칙 후보 생성	<code>scripts/synergy-candidates.json</code>
코드 리뷰 에이전트	구현 후 별도 컨텍스트에서 리뷰 (같은 세션 self-approve 금지)	커밋 <code>7b0aaf6</code> 이 지적 17건 반영

`gen-synergies.mjs`는 두 입력 모드를 지원한다. `OPENAI_API_KEY`가 있으면 OpenAI API를 호출하고, 없으면 세션에서 생성한 후보 파일을 읽는다. **어느 쪽이든 검증기는 동일하게 적용된다.** 이번 제출은 후자로 만들었다.

4. 주요 프롬프트

`scripts/gen-synergies.mjs`의 `PROMPT` 상수 전문이다. 허용 트리거와 효과 범위는 코드의 `TRIGGERS · EFFECT_RANGE`에서 문자열 보간으로 생성되므로, **스키마를 바꾸면 프롬프트가 자동으로 따라간다.**

너는 픽셀 고양이 오토배틀러의 시너지 규칙 디자이너다.

아래 스키마를 정확히 지키는 JSON 배열만 출력해라. 설명 문장은 쓰지 마라.

허용 trigger (이것 외에는 절대 쓰지 마라):

- same_color_3
- same_breed_2
- all_different_5

허용 effect.key와 값 범위 (범위를 벗어나면 잘린다):

- atk_mul: 1.1 ~ 1.6
- hp_mul: 1.1 ~ 1.6
- evade_add: 0.05 ~ 0.3
- atkspd_mul: 1.1 ~ 1.5

각 항목 형식:

```
{ "id": "소문자_스네이크", "name": "16자 이내 한글 이름",  
  "desc": "48자 이내 한 줄 설명", "trigger": "...",  
  "effect": { "key": "...", "value": 숫자 } }
```

규칙:

- trigger마다 최소 3개씩, 총 24개를 만들어라.
- id는 영문 소문자로 시작하고 중복되면 안 된다.
- 이름과 설명은 고양이답고 짧게. 수치를 설명에 쓰지 마라.

프롬프트에 범위를 적어 두지만 **지켜질 것이라고 가정하지 않는다**. 아래 검증기가 실제 계약이다.

5. 검증기 설계

src/validate/synergy-schema.ts . 빌드타임 스크립트와 런타임이 **같은 모듈**을 공유한다.

5.1 계층

계층	동작	위반 시
구조	객체인지, 필수 키가 있는지	폐기
id	<code>^[a-z][a-z0-9_]{1,31}\$</code> , 배치 내 중복 없음	폐기
trigger	TRIGGERS 화이트리스트	폐기
effect.key	EFFECT_KEYS 화이트리스트	폐기
effect.value	유한한 수	폐기
effect.value	EFFECT_RANGE 로 하드 클램프	폐기 아님 — 경계로 누름
name/desc	제어문자·꺾쇠·유니코드 서식문자 제거, 코드포인트 단위 절단	새니타이즈
name/desc	새니타이즈 후 비었는지	폐기

범위 초과를 폐기가 아니라 클램프로 처리하는 이유: 모델이 낸 아이디어(트리거 조합과 이름)는 살리고 수치만 안전하게 만드는 편이 콘텐츠 다양성에 유리하다. 반면 존재하지 않는 트리거나 효과 키는 의미를 복구할 방법이 없으므로 폐기한다.

절단을 코드포인트 단위로 하는 이유: `String.prototype.slice` 는 UTF-16 코드유닛 기준이라 이모지의 서로게이트 쌍을 반토막 낸다. 짝 잃은 서로게이트는 캔버스에 두부 글자로 렌더된다. `[...str]` 로 코드포인트 배열을 만든 뒤 자른다.

서식문자를 제거하는 이유: U+202E(RTL override)가 이름에 섞이면 `fillText` 가 뒤따르는 글자를 역순으로 그린다. XSS는 아니지만 정확히 "렌더 깨짐"이다.

5.2 적대 테스트

`npm test` — `scripts/synergy-adversarial.json` 의 규칙 위반 입력을 검증기에 통과시켜 실제로 막히는지 확인한다.

적대 입력 11개 → 폐기 8개, 통과 3개

폐기된 항목:

quad_color	허용되지 않은 trigger: same_color_4
finisher	허용되지 않은 effect.key: instant_kill
9bad_id	id 형식 위반
dup_id	id 중복: dup_id
stringy	effect.value가 유효한 수가 아님
no_effect	effect가 객체가 아님
nan_value	effect.value가 유효한 수가 아님
(문자열 항목)	객체가 아님

통과했지만 값이 늘린 항목:

dup_id	name="중복 원본" hp_mul=1.2	(중복 중 첫 번째만 생존)
tag_soup	name="태그scriptalert(1)"	(꺾쇠 제거)
overshoot	atk_mul=1.6	(원본 9.9 → 상한 클램프)

전부 통과 – 검증기가 적대 입력을 모두 막거나 안전한 값으로 늘렸다.

테스트는 네 가지를 단언한다: 통과분의 값이 전부 허용 범위 안일 것, 이름·설명에 꺾쇠가 없을 것, 명백히 불법인 항목이 통과하지 않을 것, 범위 초과는 폐기가 아니라 상한으로 늘릴 것.

5.3 폴백

`resolveSynergyPool()` 은 검증 통과분이 3개 미만이면 하드코딩된 프리셋으로 채운다.

`src/data/synergies.json` 을 빈 배열로 만들어 빌드해도 게임이 정상 동작하고 프리셋 시너지(야행성 / 한배 형제 / 잡색 부대)로 플레이된다 — 실제로 확인했다.

6. AI가 만든 규칙을 사람이 아니라 시뮬레이션으로 검증한다

생성된 규칙이 밸런스를 깨는지는 눈으로 알 수 없다. 그래서 게임 로직을 브라우저 없이 그대로 돌리는 하네스를 만들었다. `scripts/balance-sim.mjs` 는 별도 구현이 아니라 `stepBattle` · `buyOffer` 를 직접 임포트하므로 **시뮬과 실제 게임이 갈라지지 않는다**.

```
$ npm run sim
런 300회 (정책: 살 수 있으면 가장 비싼 것 구매, 재배치 없음)
최소 8 p25 10 중앙값 10 p75 10 최대 15
평균 10.2웨이브
판정: 목표 구간(6~20) 안
```

이 하네스가 실제로 잡아낸 결함:

증상	원인	수정
웨이브 2에서 전멸	적은 웨이브마다 복리로 크는데 플레이어 레벨 성장은 선형	레벨 성장을 지수로
12%가 60웨이브 상한 도달, 런이 안 끝남	강화 효과는 지수인데 비용이 선형	비용도 지수로
모든 파라미터 조합이 중앙값 5로 동일	보드 만석 시 영입 오퍼가 목록에 남아 무한 재시도	불가능한 오퍼는 즉시 제거
탱키 조합이 매 웨이브 타임아웃 승리	타임아웃을 남은 체력 비율로 판정	타임아웃은 패배

마지막 항목은 생성 규칙과 직접 관련이 있다. `evade_add` 가 겹친 조합이 아무도 죽이지 못하면서 시간만 끄는 상태가 나왔고, 이는 LLM이 만든 규칙 조합이 만들어낸 상태 공간이었다. 검증기가 개별 규칙의 문법을 보장하더라도 **조합의 결과는 시뮬로만 알 수 있다.**

전투에 이동을 넣은 뒤(근접이 걸어가서 붙고 원거리는 제자리에서 쏘는 구조) 같은 하네스를 다시 돌려 도달 웨이브 분포가 유지되는지 확인했다. 전투 길이가 1~2초 늘어 타임아웃 상한을 12초에서 16초로 올린 것 외에 밸런스 파라미터는 건드리지 않았다. **게임 로직을 바꿀 때마다 눈이 아니라 이 분포를 본다**는 것이 이 프로젝트의 규칙이다.

추가로 `pickSynergies` 는 트리거당 최대 1개만 뽑는다. 같은 트리거의 규칙이 여러 개 켜지면 한 조합으로 배수가 중첩되어 밸런스가 무너지기 때문이다.

7. 그 밖의 AI 활용

- **스프라이트 파이프라인:** 캐릭터 시트 PNG 한 장(1374×4306)에서 20종 × 6포즈를 프로그래매틱하게 잘라낸다 (`scripts/slice-sprites.py`). 배경 제거는 테두리에서 시작하는 flood fill로만 한다 — 전역 색상 치환을 쓰면 크림색·흰색 고양이 몸통까지 지워지기 때문이다.
- **레이아웃 회귀 검사:** `scripts/layout-probe.mjs` 가 기기 10종에서 보상 카드 패널과 보드 셀의 겹침 면적을 계산한다. 실제로 이 프로브가 "세로 화면에서 아랫줄 3칸이 가려지고 탭까지 가로채는" 결함을 수치로 확인해 주었다.
- **리뷰 분리:** 구현한 컨텍스트에서 스스로 승인하지 않고, 별도 리뷰 에이전트가 지정한 17건(치명 1, 높음 4, 중간 6, 낮음 6)을 반영했다.

8. 외부 에셋 · 오픈소스 출처

항목	출처	라이선스
고양이 스프라이트 (20종 × 6포즈)	자체 제작 (mmporong, 2026)	자체 보유
TypeScript	Microsoft	Apache-2.0
Vite	Vite 팀	MIT
폰트	OS 시스템 폰트 (system-ui)	별도 임베드 없음
사운드	사용하지 않음	—

런타임 의존성은 **0개**다. 게임 엔진, 상태관리 라이브러리, UI 프레임워크를 쓰지 않았다. 빌드 산출물은 JavaScript 약 25 kB(gzip 10 kB)와 스프라이트 PNG다.

타인의 저작물이나 아이디어를 무단 사용한 부분은 없다.

9. 재현 방법

```
git clone https://github.com/mmporong/nyang-arena.git
cd nyang-arena
npm install

npm test           # 검증기 적대 테스트
npm run probe      # 기기 10종 레이아웃 겹침 검사
npm run sim         # 밸런스 시뮬레이션 300런
npm run gen:synergies # 시너지 규칙 재생성 (키 있으면 API, 없으면 세션 후보)
npm run build       # 타입체크 + 프로덕션 빌드
```